

Presentación del Curso

 Este curso forma científicos de datos y profesionales técnicos en tecnologías de IA generativa de última generación, con énfasis en sistemas RAG, agentes inteligentes y conectividad avanzada con bases de datos.

 Los participantes aprenderán a crear aplicaciones analíticas conversacionales, implementar procesamiento documental mediante **OCR avanzado**, y desarrollar **agentes autónomos** capaces de realizar investigación profunda.

Lo que te llevas

🌟 En 4 semanas construirás un ecosistema completo de herramientas de IA generativa que automatizarán tus procesos analíticos.

⚡ Te convertirás en el referente técnico de tu organización en tecnologías de IA generativa aplicadas a datos.

🤝 **Networking exclusivo** con profesionales y expertos del ecosistema IA.

Entregables:

-  **Arquitecturas empresariales** funcionales
-  **Scripts y documentación técnica** completa
-  **Portfolio de agentes** especializados
-  **Sistemas conectividad DB-LLM**
-  **Red de contactos** especializados en IA
-  **Certificado** Universidad de los Andes

Metodología

 **Virtual Sincrónica - Enfoque "Arquitectura Primero"** Cada sesión resulta en un sistema funcional y escalable

Estructura de cada módulo:

- **30% Teórico-arquitectónico:** Principios, frameworks y casos empresariales
- **70% Práctico-implementativo:** Talleres intensivos en Google Colab

Stack tecnológico enterprise:

- Google Colab, Groq API, Ollama
- OCR Avanzado (Mistral, Reducto)
- CrewAI, LangChain, AutoGen
- Model Context Protocol (MCP)



Módulo 1: Fundamentos (5h)



Semana 1 - Configuración del ecosistema

Martes (2.5h): Fundamentos Teóricos

- Introducción a IA aplicada a ciencia de datos
- NLP: embeddings y búsquedas semánticas
- Arquitecturas Transformers y Diffusion Models

Jueves (2.5h): Stack Tecnológico

- Google Colab: configuración avanzada
- Ecosistema Hugging Face
- Setup APIs: OpenAI, Anthropic, Groq
- Taller: primer modelo funcional



Módulo 2: GenBI Conversacional (5h)



Semana 2 - Inteligencia de negocios generativa

Martes (2.5h): RAG y Fine-Tuning

- RAG: arquitectura y recuperación
- Fine-tuning: LoRA, QLoRA para datos tabulares
- Implementación con datasets empresariales

Jueves (2.5h): Sistema GenBI

- Arquitectura Business Intelligence generativo
- Motor consultas: lenguaje natural → SQL/Python
- Interfaces con Chainlit y Gradio
- Dashboard conversacional completo



Módulo 3: OCR y Conectividad (5h)



Semana 3 - Del documento a la base conversacional

Martes (2h): OCR Nueva Generación

- Herramientas: Tesseract, Mistral, Reducto
- Pipeline: facturas, contratos, documentos complejos
- Extracción de tablas y estructuras

Jueves (2h): Model Context Protocol

- MCP: arquitectura y configuración
- Conectores SQL/NoSQL
- LLMs como interfaz para consultas complejas



Espacio para proyecto final: 1h total distribuida



Módulo 4: Agentes y Proyecto (5h)



Semana 4 - Automatización con agentes autónomos

Martes (2.5h): Frameworks de Agentes

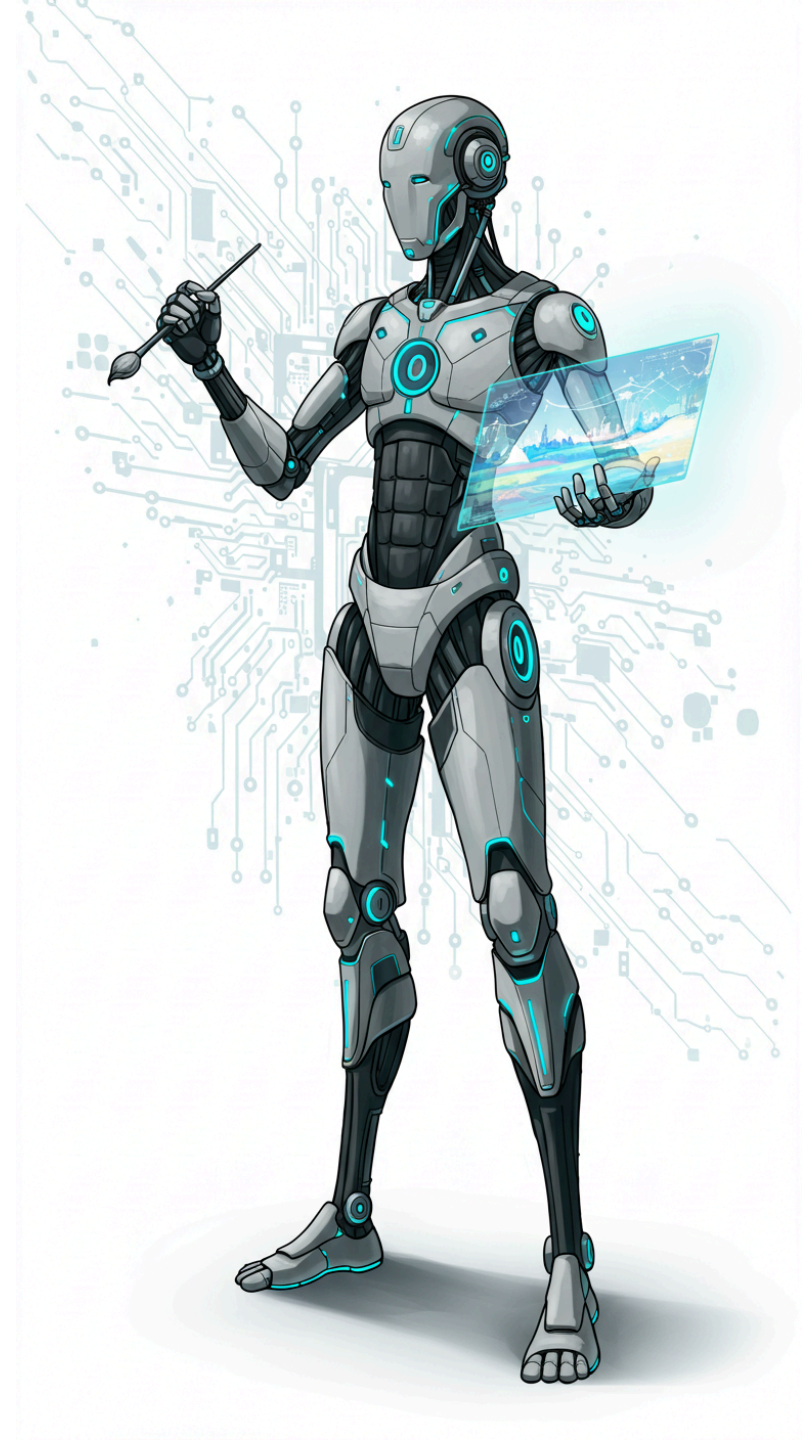
- Arquitectura de agentes analíticos
- LangChain: chains, tools, memory
- CrewAI: multi-agente y workflows
- AutoGen: conversaciones multi-agente

Jueves (2.5h): Proyecto Final

- Desarrollo grupal integrando todas las técnicas
- Presentación de proyectos
- Entrega de certificados y portfolio

Introducción a la IA Generativa

Historia, NLP y Arquitecturas
Modernas





Historia de la Inteligencia Artificial

Orígenes Ancestrales

La fascinación por crear seres artificiales se remonta a la antigua Grecia:

- **Autómatas de Herón:** Máquinas que podían moverse solas, abrir puertas de templos y servir vino
- **Talos:** El gigante de bronce de la mitología griega, primer "robot" de la literatura
- **Galatea:** La estatua que cobra vida, reflejo del sueño de crear vida artificial



El test de Turing 🎯

"¿Pueden pensar las máquinas?" - Alan Turing, 1950

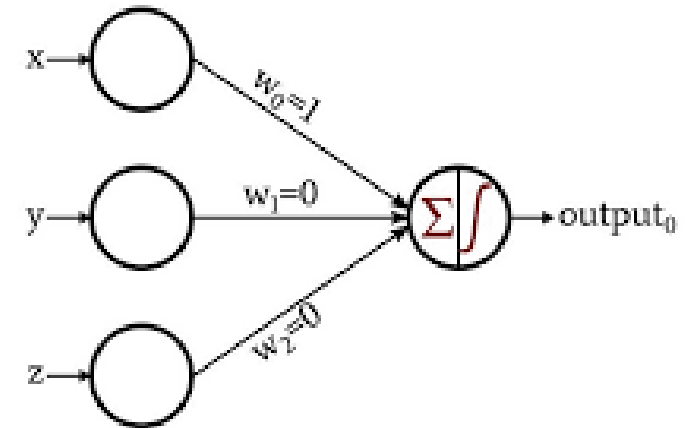
El Test de Turing propuso una idea revolucionaria: si no podemos distinguir entre las respuestas de una máquina y un humano, entonces la máquina demuestra inteligencia. Este principio fundamental sigue guiando el desarrollo de la IA conversacional moderna.



El Perceptrón 🤖

Frank Rosenblatt creó en 1957 el Perceptrón, inspirado en el cerebro humano. Este primer intento de crear redes neuronales artificiales prometía revolucionar la computación, pero sus limitaciones llevaron al primer "invierno de la IA":

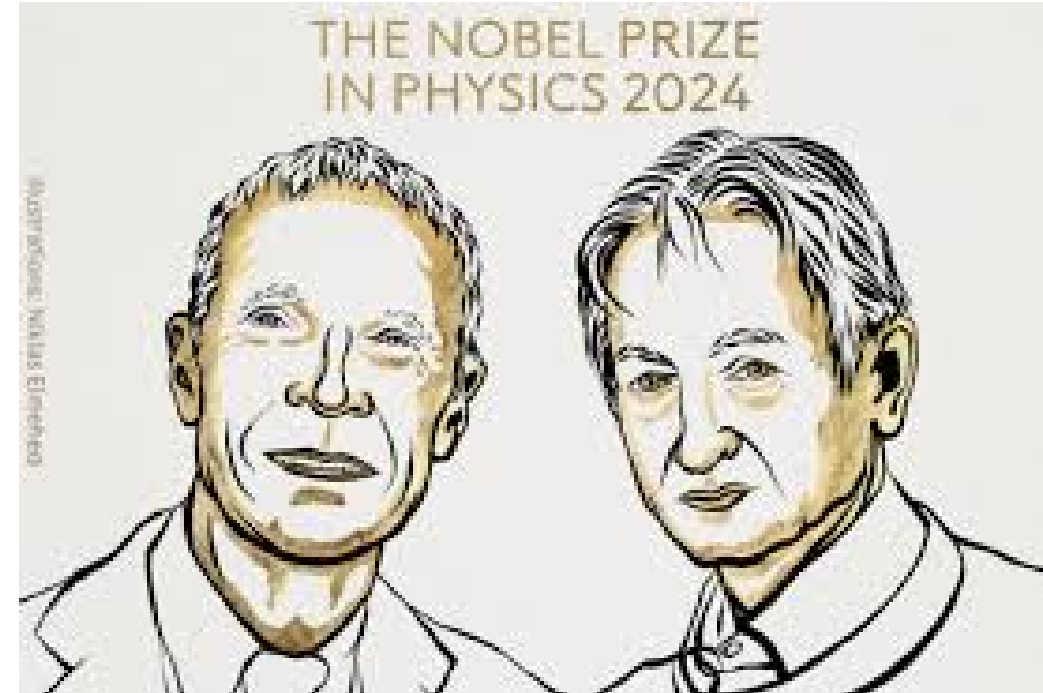
- 📊 No resolvía problemas no lineales
- 🔬 Necesitaba un nuevo enfoque
- ❄️ Provocó un pausa en la investigación



El aporte de los nobel 🧠

Los 80s marcaron un punto de inflexión:

- 🎯 **1982:** John Hopfield revoluciona el campo con sus redes recurrentes, demostrando que las redes neuronales podían tener memoria y resolver problemas complejos
- ⚡ **1986:** Geoffrey Hinton introduce la retropropagación y las redes profundas, permitiendo entrenar sistemas multicapa
- 🚀 Pero, los avances requerían mayor poder de computo



El aporte Gaming 🎮

Los videojuegos cambiaron el juego: las GPU, diseñadas para gráficos 3D, revolucionaron el entrenamiento de redes neuronales. Esta tecnología de procesamiento en paralelo resultó ideal para la IA.

2006: NVIDIA lanza CUDA, transformando el panorama del deep learning y potenciando una nueva generación de investigación en IA.

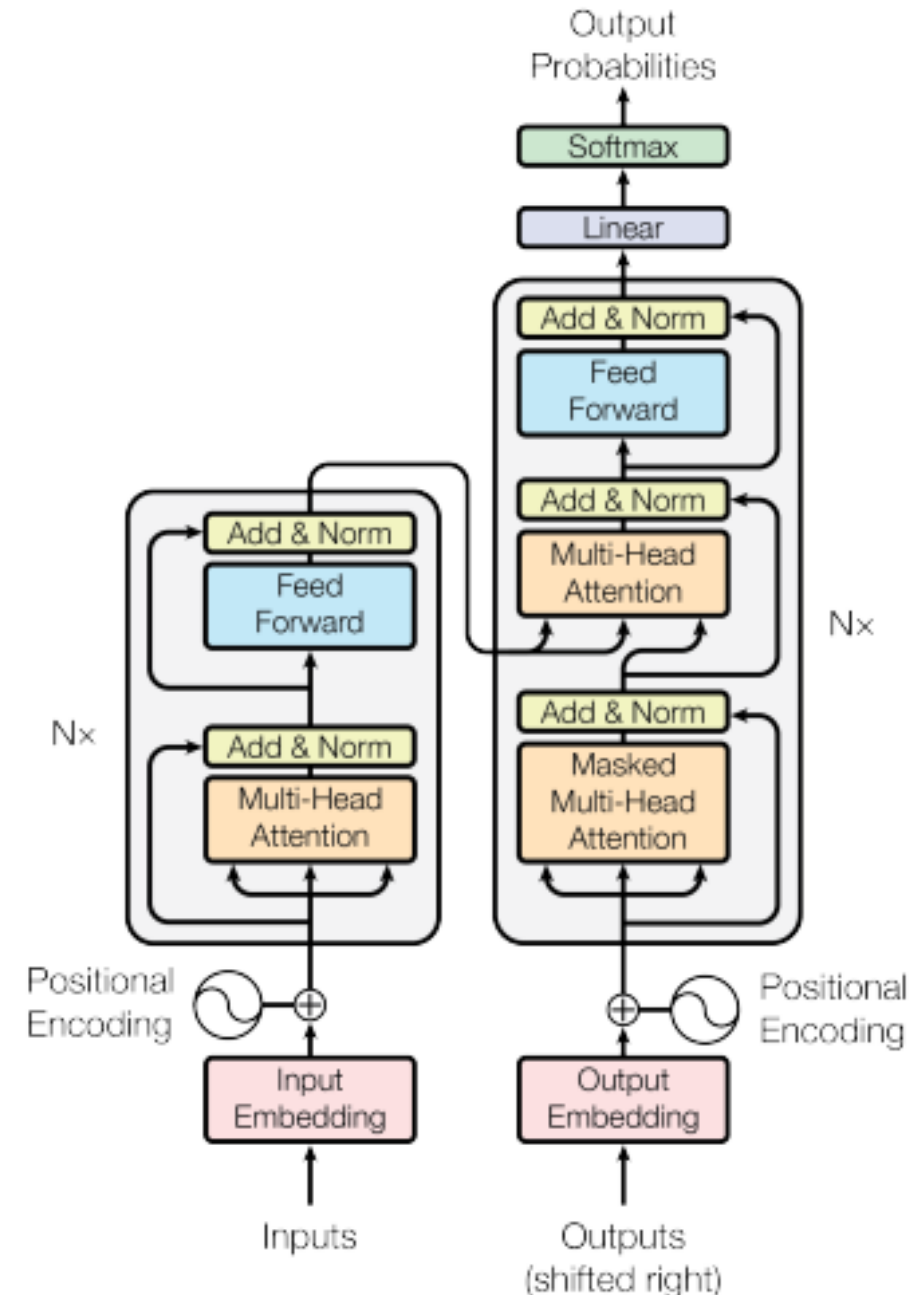


La revolución Actual 🌐

2017: Los Transformers revolucionan el procesamiento del lenguaje, estableciendo las bases para modelos como GPT y BERT.

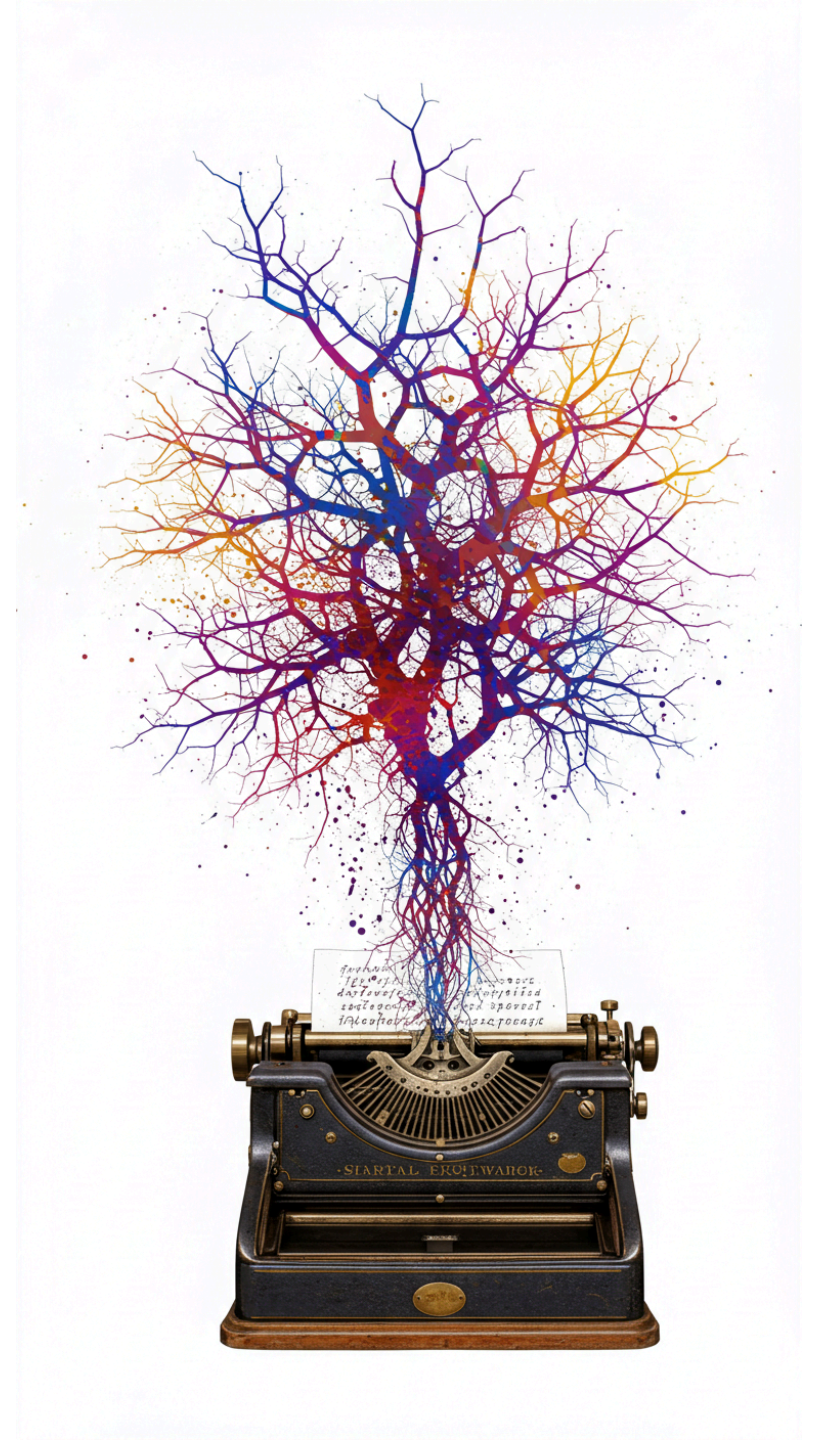
Hoy vivimos la era de:

- 🤖 IA generativa cada vez más sofisticada
- 🎨 Modelos multimodales que integran diversos tipos de datos
- 🌍 Democratización de la tecnología
- 🧠 Innovación continua en el campo



Fundamentos de Procesamiento de Lenguaje Natural (NLP)

Hablando con la máquina  



Introducción al Procesamiento de Lenguaje Natural

El Procesamiento de Lenguaje Natural (NLP) es una rama de la inteligencia artificial que permite a las máquinas entender 📖, interpretar 🔍 y generar ✍️ lenguaje humano. Su objetivo principal es cerrar la brecha comunicativa entre humanos y computadoras.

El NLP combina lingüística computacional , aprendizaje automático 🤖 y análisis estadístico  para procesar texto y voz de forma significativa. Esto permite aplicaciones como asistentes virtuales, traducción automática, análisis de sentimiento y extracción de información.

¿Para qué sirve el NLP?

-  Asistentes virtuales y chatbots inteligentes
-  Análisis de sentimiento en texto
-  Resumen automático de documentos
-  Traducción automática entre idiomas
-  Sistemas de búsqueda semántica
-  Extracción de información estructurada
-  Publicidad y marketing personalizado
-  Interfaces conversacionales

El Reto del Preprocesamiento de Textos

Para que las máquinas entiendan texto, primero debemos transformarlo en algo procesable .

El preprocesamiento determina cómo el modelo "ve" e interpreta el lenguaje, condicionando todo el funcionamiento posterior.

Es como enseñar a leer a un niño .

1. Primero aprende letras 
2. Luego forma palabras 
3. Después comprende oraciones 
4. Finalmente interpreta significados 

Tokenización: Fundamentos

La tokenización es el proceso de dividir texto en unidades básicas llamadas tokens  que pueden ser procesadas por modelos computacionales. Este paso fundamental determina cómo los sistemas de NLP interpretarán el lenguaje.

El proceso varía según idioma y aplicación . En lenguas con separación clara como el español o inglés, los espacios sirven como delimitadores naturales. Sin embargo, en idiomas como el chino o japonés sin espacios, o con morfología compleja como el finlandés, la tokenización requiere enfoques especializados.



Normalización de Texto

Técnicas comunes

- Case folding : Convertir a minúsculas
 - "*Economía*" → "*economía*"
- Eliminación de stopwords :
 - Remover palabras muy frecuentes con poco valor semántico
 - "*el*", "*la*", "*y*", "*que*", "*en*", "*a*"
- Eliminación de puntuación y caracteres especiales 
 - "*economía!*" → "*economía*"
- Corrección ortográfica 
 - "*ecnomia*" → "*economía*"

Stemming vs Lemmatization

Ejemplo práctico:

- Stemming: "corrieron", "correrá", "corredores" → "corr" 
- Lemmatization: "corrieron", "correrá", "corredores" → "correr" 

Stemming

- Proceso más rápido 
- Basado en reglas heurísticas 
- Mayor tasa de error 
- Ideal para grandes volúmenes 

Lemmatization

- Proceso más lento 
- Basado en diccionarios 
- Mayor precisión 
- Ideal para análisis lingüístico 



Word Embeddings: Semántica en el Espacio Vectorial

Los word embeddings son representaciones vectoriales de palabras en un espacio multidimensional  donde la posición y distancia entre vectores captura relaciones semánticas. Esta técnica revolucionó el NLP al permitir que los modelos comprendan significados y similitudes entre palabras.

A diferencia de las representaciones tradicionales (como one-hot encoding), los embeddings capturan relaciones semánticas: palabras con significados similares tienen vectores cercanos en el espacio .

La propiedad más notable es la posibilidad de realizar operaciones algebraicas con significados : "Rey - Hombre + Mujer $\approx$ Reina"

Word Embeddings: Propiedades Fundamentales

Los word embeddings transforman palabras en vectores multidimensionales con propiedades semánticas útiles:

- La **posición** en el espacio refleja el significado 📍
- La **distancia** entre vectores cuantifica similitud semántica 📏
- Las **direcciones** capturan relaciones conceptuales 🧭

Operaciones semánticas

- Similitud: $\cos(\text{economía}, \text{finanzas}) \approx 0.8$
- Analogías: París:Francia :: Madrid:España
- Agrupamiento: formación de clusters semánticos

Aplicaciones principales

- Búsqueda semántica 🔍
- Sistemas de recomendación 👍
- Análisis de sentimiento 😊
- Base para modelos avanzados 🤖

Similitud de Coseno: Matemática de la Semántica

La similitud de coseno es la métrica fundamental para medir qué tan "cercanas" semánticamente están dos palabras en el espacio de embeddings.

$$\text{similitud_coseno}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \sqrt{\sum_{i=1}^n b_i^2}}$$

Donde:

- $\vec{a}$ y $\vec{b}$ son vectores de embedding 
- $\vec{a} \cdot \vec{b}$ es el producto escalar ✕
- $|\vec{a}|$ y $|\vec{b}|$ son las magnitudes (normas) 
- El resultado varía entre -1 (direcciones opuestas) y 1 (misma dirección) 

Ejemplo Práctico: Similitud de Coseno

Vectores de embedding simplificados (dimensión reducida):

- $\vec{economía} = [0.2, 0.5, -0.3, 0.8]$
- $\vec{finanzas} = [0.3, 0.6, -0.2, 0.7]$
- $\vec{literatura} = [-0.4, 0.1, 0.7, 0.2]$

Resultados comparativos

- $\text{sim}(\text{economía}, \text{finanzas}) = 0.98 \rightarrow$ ¡Muy similares! 
- $\text{sim}(\text{economía}, \text{literatura}) = -0.22 \rightarrow$ Divergentes 
- $\text{sim}(\text{finanzas}, \text{literatura}) = -0.05 \rightarrow$ Casi ortogonales $\perp$

La similitud de coseno mide el **ángulo** entre vectores, ignorando sus magnitudes. Valores cercanos a 1 indican gran similitud semántica.

Dimensionalidad en Word Embeddings

Los embeddings de alta dimensionalidad permiten capturar relaciones semánticas más complejas entre palabras 🧠.

Servicios de Embeddings 🚀

- **Word2Vec/GloVe**: 50-300 dimensiones (estándar)
- **FastText**: 300 dimensiones (común)
- **BERT**: 768-1024 dimensiones (capas profundas)
- **Ada (OpenAI)**: 1536 dimensiones (estado del arte)

El aumento de dimensiones tiene un límite práctico donde el beneficio marginal se reduce frente al costo computacional y riesgo de sobreajuste.

Arquitecturas disruptivas en la IA Generativa

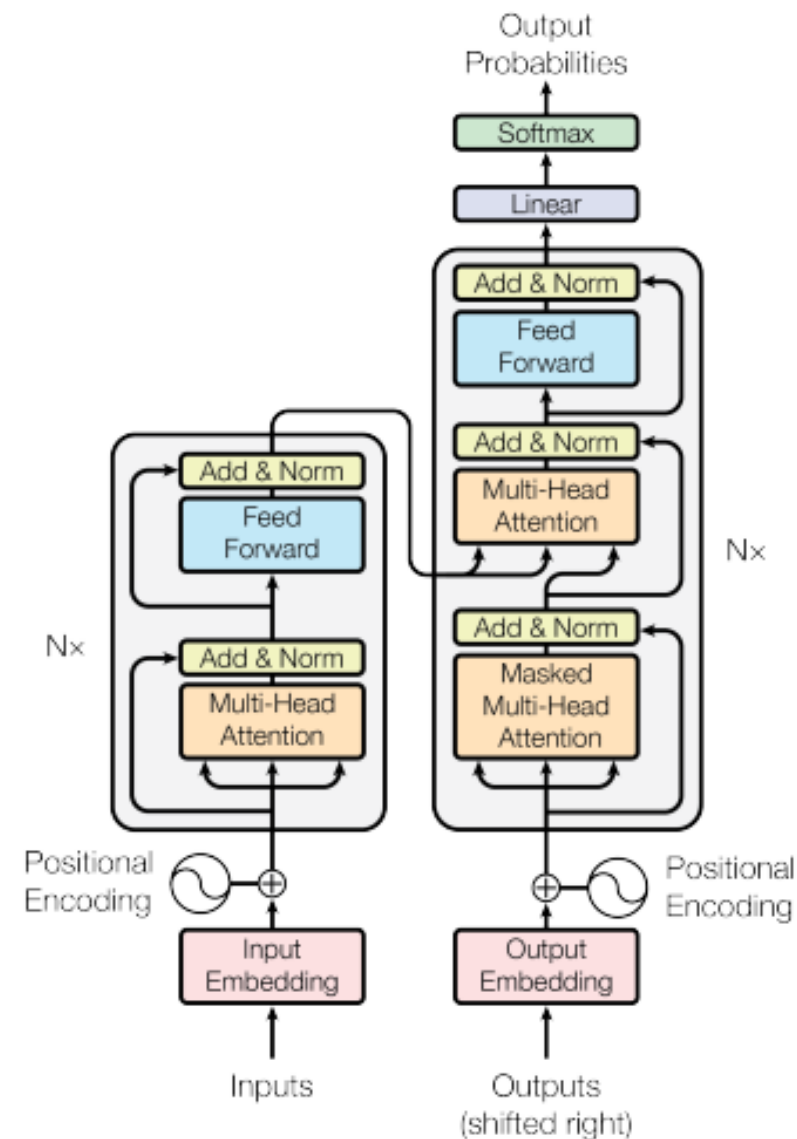
Transformers y Diffusers



La disrupción de los Transformers

La arquitectura Transformer, introducida en 2017 en el paper "**Attention is All You Need**", revolucionó la IA al permitir que los modelos comprendan profundamente el contexto y generen contenido coherente y creativo a escala sin precedentes.

A diferencia de sistemas anteriores, los Transformers pueden "ver" todas las palabras simultáneamente y establecer conexiones directas entre ellas, sin importar la distancia. Esta capacidad ha dado origen a tecnologías como ChatGPT, DALL-E y Midjourney.





Transformer vs RNN: Una nueva forma de procesar secuencias



RNN (Arquitectura Previa)

- Procesamiento **secuencial** (palabra por palabra)
- Limitación de **memoria a largo plazo**
- **Lento entrenamiento** (no paralelizable)
- Dificultad con **contextos amplios**



Transformer

- Procesamiento **paralelo** (todas las palabras a la vez)
- **Conexión directa** entre cualquier par de palabras
- **Rápido entrenamiento** en GPUs
- Captura **relaciones complejas** entre palabras distantes (**Atención**)

El mecanismo de atención permite que cada palabra "preste atención" a todas las demás palabras simultáneamente, creando representaciones contextualizadas mucho más ricas.

Con Embeddings de posición sobran las RNN

Embeddings Posicionales

- A diferencia de las RNN, los Transformers **no procesan secuencialmente**
- Necesitan saber **el orden de las palabras**
- Los embeddings posicionales **codifican la posición** de cada token
- Utilizan funciones sinusoidales para mantener relaciones posicionales

"Attention Is All You Need"

- Título revolucionario que significaba: "**no necesitamos RNNs**"
- Reemplaza recurrencia con **conexiones directas** entre palabras
- **Elimina** el procesamiento secuencial obligatorio
- Permite **paralelización masiva** en hardware moderno

```
PE(pos, 2i) = sin(pos/10000^(2i/d_model))  
PE(pos, 2i+1) = cos(pos/10000^(2i/d_model))
```

Mecanismo de Atención: Conceptos básicos

La atención es como un "foco de luz" que permite al modelo concentrarse en partes relevantes de la entrada.

- **Query (Q):** "¿Qué estoy buscando?"
 - Como hacer una pregunta específica en una biblioteca
- **Key (K):** "¿Qué información ofrezco?"
 - Como las etiquetas o índices de los libros
- **Value (V):** "¿Qué contenido apporto?"
 - El contenido real de cada libro
- **Atención:** El proceso de encontrar los libros más relevantes (K) para tu búsqueda (Q) y extraer su contenido valioso (V)

1 2
3 4

Mecanismo de Atención: Formalización Matemática



Componentes Matemáticos

- Query (Q): Lo que estamos buscando
- Key (K): Lo que podría coincidir con nuestra búsqueda
- Value (V): La información que queremos extraer

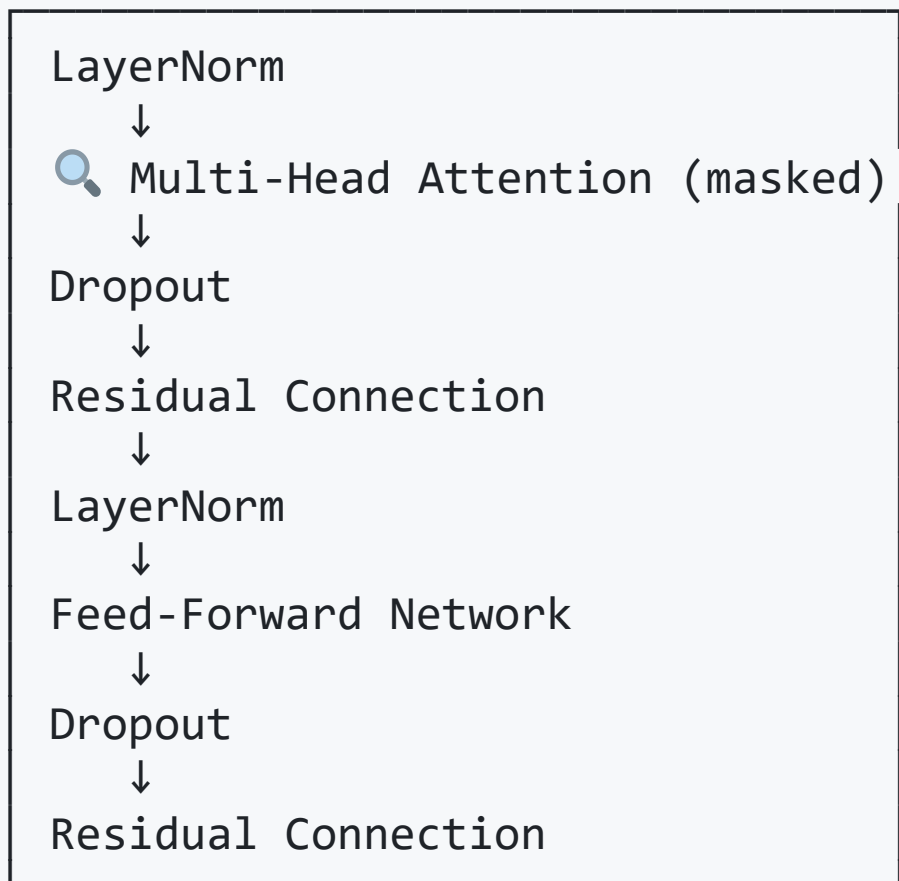


Fórmula de Atención

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

La atención calcula cuánto debe "mirar" cada token a todos los demás, luego combina la información de manera ponderada.

Multi-Head Attention: El corazón del Transformer



Multi-Head Attention (MHA)

- Permite al modelo **enfocar** diferentes aspectos del contexto simultáneamente
- Opera con **atención causal/enmascarada** (cada token solo ve tokens anteriores)
- Cada cabeza de atención aprende diferentes tipos de relaciones:
 - **Sintácticas**: estructura gramatical
 - **Semánticas**: significado y contexto
 - **Referencias**: conexiones entre entidades

Re-interpretaciones del Transformer

El Transformer original de 2017, aunque revolucionario, era **complejo y computacionalmente costoso** 🏗️ por su arquitectura de codificador-decodificador. Los investigadores pronto descubrieron que podían obtener resultados sorprendentes utilizando **solo una de sus mitades** ✂️, lo que dio origen a dos familias de modelos especializados.

Por un lado, modelos como **BERT** 🔍 aprovecharon exclusivamente el **codificador** para tareas de comprensión, empleando **atención bidireccional**. Por otro lado, **GPT** ✍️ optó por usar solo el **decodificador** con atención unidireccional para destacar en generación. Esta **bifurcación evolutiva** 🌱 marcó el inicio de una carrera para determinar cuál de estos enfoques simplificados alcanzaría mayores capacidades.



De Transformer a GPT: El inicio de una nueva era

OpenAI y el Generative Pre-trained Transformer (2018)

OpenAI dio un paso transformador en 2018 al introducir GPT (Generative Pre-trained Transformer), una simplificación brillante del modelo Transformer original.

A diferencia del Transformer completo, GPT utilizó únicamente la parte del **decodificador**, descartando el codificador. Esta decisión de diseño resultó ser una revolución en el procesamiento del lenguaje natural.

El término "GPT" nace como acrónimo de "Generative Pre-trained Transformer", señalando el enfoque en la generación de texto mediante pre-entrenamiento a gran escala.



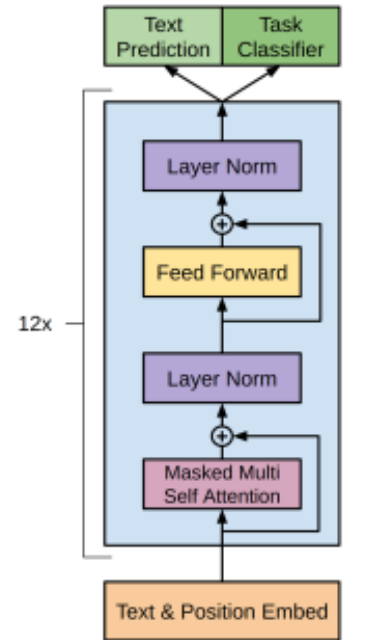
Radford et al. (2018) - ["Improving Language Understanding by Generative Pre-Training"](#)

🔑 La innovación clave de GPT-1

La principal contribución de GPT-1 fue su estrategia de entrenamiento en dos etapas:

1. **Pre-entrenamiento generativo:** El modelo aprende a predecir la siguiente palabra en textos sin etiquetar (BookCorpus, 7,000+ libros)
2. **Fine-tuning supervisado:** Adaptación del modelo pre-entrenado a tareas específicas con mínimos cambios arquitectónicos

Este enfoque permitió aprovechar enormes cantidades de texto sin necesidad de anotaciones costosas.



Arquitectura de GPT-1 - 12 capas de decodificador - 117M parámetros - Atención causal (autoregresiva) - Predice tokens futuros solo a partir de contexto previo

BERT: El enfoque alternativo solo-codificador

Google y la revolución bidireccional (2018)

Casi simultáneamente a GPT-1, Google presentó BERT con enfoque distinto:

- **Solo codificador** del Transformer original
- **Atención bidireccional** - puede procesar contexto en ambas direcciones
- Nuevas tareas de pre-entrenamiento:
 - **Masked LM**: Predecir palabras ocultas
 - **Next Sentence Prediction**: ¿Son consecutivas?

A diferencia de GPT (unidireccional), BERT "mira" en ambas direcciones, creando representaciones contextuales más ricas.

Aplicaciones principales: - Clasificación de texto - Extracción de información - Respuesta a preguntas

 [Devlin et al. \(2018\) - "BERT: Pre-training of Deep Bidirectional Transformers"](#)

🤔 GPT vs BERT: Enfoques complementarios

Dos caminos desde el Transformer original

🧠 **GPT (Solo decodificador)** - Atención unidireccional (causal) - Generación de texto fluida -
Pre-entrenamiento: predecir siguiente token - Ideal para tareas generativas

🔬 **BERT (Solo codificador)** - Atención bidireccional - Comprensión profunda del contexto - Pre-entrenamiento: predecir tokens ocultos - Ideal para clasificación y análisis

Cada enfoque tiene sus ventajas: BERT excede en comprensión pero carece de capacidad generativa natural, mientras que GPT genera texto fluido pero con visión contextual limitada.

Estos dos enfoques definieron familias de arquitecturas que evolucionaron en paralelo, cada una optimizada para diferentes tipos de tareas.

GPT-2: El salto hacia las capacidades emergentes

"Language Models are Unsupervised Multitask Learners" (2019)

Un año después, OpenAI sorprendió al mundo con GPT-2, un modelo que representó un salto cuántico en escala y capacidades:

- **Escalamiento masivo:** De 117M a 1.5B parámetros (10× mayor)
- **WebText:** Nuevo dataset curado de alta calidad (45M enlaces)
- **Zero-shot learning:** Capacidad para realizar tareas sin entrenamiento específico

La innovación más sorprendente fue el descubrimiento de que el simple escalamiento del modelo y los datos produjo "capacidades emergentes" - habilidades que nadie había programado explícitamente.

💡 Las habilidades emergentes del GPT-2

El poder inesperado del escalamiento

GPT-2 demostró capacidades que nadie había anticipado en un modelo sin entrenamiento específico:

- **Traducción inglés-francés** sin haber visto pares de traducción
- **Resúmenes coherentes** sin entrenamiento en resumen
- **Respuestas a preguntas factuales** sin una base de conocimiento explícita
- **Generación de texto** manteniendo estilo y contexto en párrafos largos

Descubrimiento fundamental

Los investigadores observaron que el modelo había adquirido capacidades que tradicionalmente requerían arquitecturas específicas y entrenamiento supervisado.

Como señalaron en el paper:

"Demostramos que los modelos de lenguaje comienzan a aprender estas tareas sin supervisión explícita."

Datos de Entrenamiento de GPT-3

Composición del corpus (Raschka, "LLM from Scratch")

Fuente	Descripción	Proporción	Tokens
Common Crawl	Web crawl data	60%	410B
WebText2	Web crawl data	22%	19B
Books1 y Books2	Internet-based book corpus	16%	67B
Wikipedia	High-quality text	3%	3B

499 mil millones de tokens en total, con énfasis en la calidad y diversidad.



De GPT-3 a ChatGPT: La revolución conversacional

Evolución hacia asistentes IA alineados

🔍 A pesar de sus impresionantes capacidades, los modelos GPT originales generaban texto **sin entender realmente las necesidades humanas**. En 2022, OpenAI dio un paso revolucionario al transformar GPT en ChatGPT, convirtiendo un modelo generativo en un **asistente conversacional útil y seguro**.

⚙️ Esta transformación fue posible gracias a dos técnicas fundamentales: el **Fine-tuning con Instrucciones Supervisadas (SFT)**, que enseñó al modelo a entender y seguir indicaciones específicas, y el **Aprendizaje por Refuerzo con Feedback Humano (RLHF) con Proximal Policy Optimization (PPO)**, que alineó el modelo con valores y preferencias humanas mediante un sofisticado sistema de votación.

🎨 Diffusion Models: El arte del ruido

📊 Los modelos de difusión representan una revolución en la generación de imágenes, transformando ruido aleatorio en contenido visual detallado y realista mediante un proceso inspirado en la termodinámica.

🔄 El principio fundamental es sorprendentemente intuitivo: primero aprenden a destruir información gradualmente añadiendo ruido a las imágenes, para luego invertir este proceso y reconstruir imágenes realistas desde el caos.





Origen y evolución de los modelos de difusión



El concepto tiene raíces en la física estadística, específicamente en los procesos de difusión donde las partículas se mueven de áreas de alta concentración a baja concentración hasta alcanzar equilibrio.



En 2015, Sohl-Dickstein y colaboradores propusieron el primer modelo de difusión para generación de imágenes, pero su adopción masiva llegó años después con el trabajo de **Ho, Jain y Abbeel (2020)** quienes introdujeron los "Diffusion Probabilistic Models" (DDPM).



La explosión de popularidad ocurrió en 2022 con el lanzamiento de Stable Diffusion, DALL-E 2 y Midjourney, llevando esta tecnología al público general y transformando industrias creativas enteras.

⚙️ ¿Cómo funcionan los modelos de difusión?

El proceso consta de dos fases fundamentales:

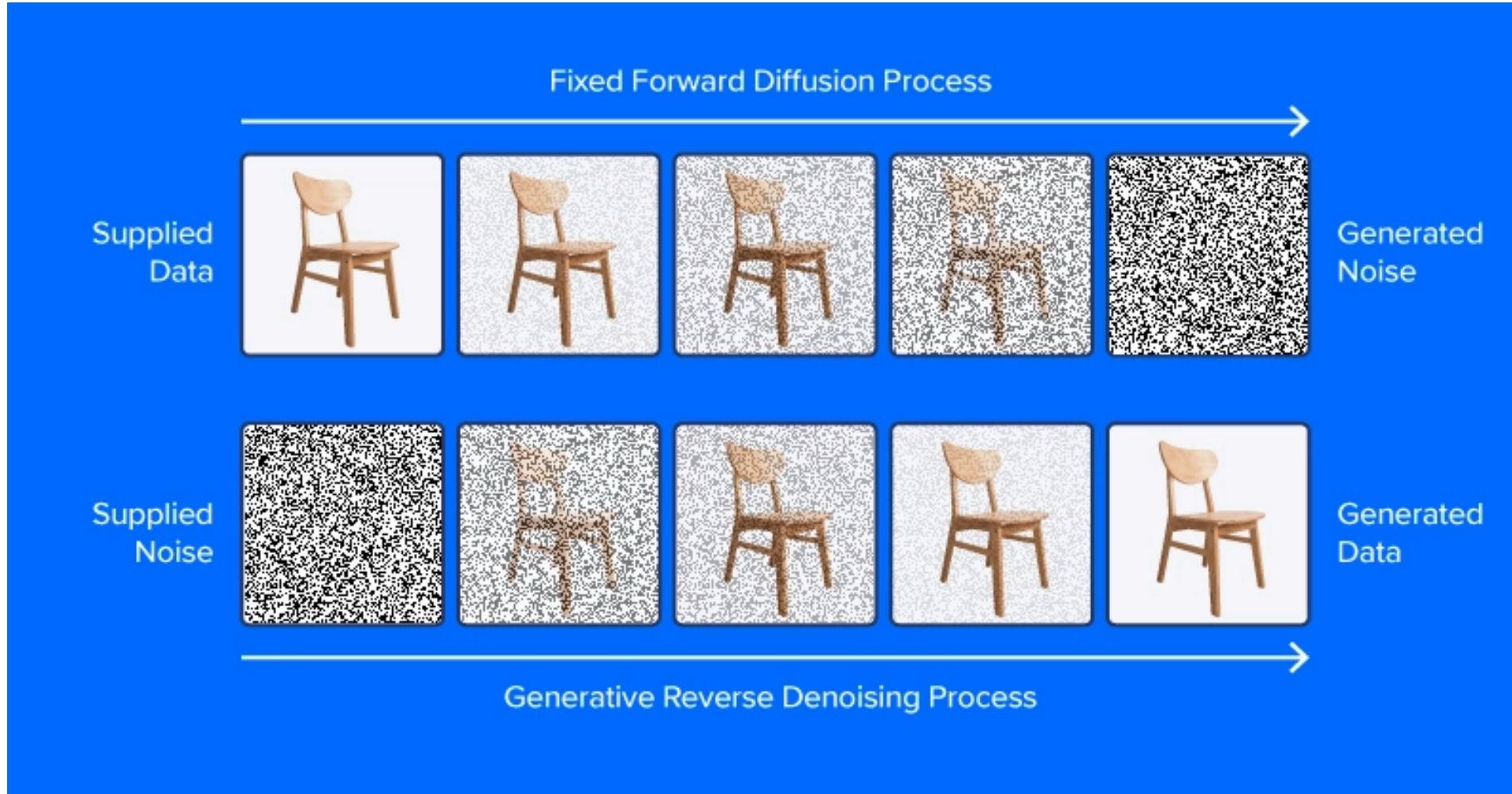
1. **Proceso de difusión directa (Forward):** Añade ruido gaussiano gradualmente a una imagen hasta convertirla en ruido puro
2. **Proceso de difusión inversa (Reverse):** Aprende a eliminar el ruido paso a paso para reconstruir la imagen

La magia está en el entrenamiento: el modelo aprende a predecir el ruido añadido en cada paso, lo que le permite posteriormente revertir el proceso con imágenes nunca vistas.

En esencia, un modelo de difusión no genera imágenes directamente, sino que aprende a "limpiar" ruido gradualmente hasta formar una imagen coherente.



El proceso de difusión en acción



Arriba: Proceso de difusión directa (Forward) que añade ruido progresivamente. Abajo: Proceso generativo inverso (Reverse) que elimina ruido para crear la imagen. Fuente: Contentstack.io

abc Text-to-Image: Guiando la difusión con palabras

Durante el entrenamiento, las imágenes con sus etiquetas crean una **base de datos de ruido etiquetado**.

Cuando escribimos un prompt como "**silla aguacate**", el modelo:

1. **Vectoriza** los conceptos "silla" y "aguacate"
2. **Combina** sus representaciones en el espacio latente
3. **Guía** el proceso de denoising para manifestar ambos conceptos en una imagen coherente



Ejemplo de "silla aguacate" generada por un modelo de difusión. La combinación de conceptos demuestra la capacidad del modelo para fusionar representaciones semánticas.

Material complementario

Repositorio del Curso IA Aplicada del profesor Camilo Vega

 Semana 11 - Arquitectura Transformers: github.com/CamiloVga/Curso-IA-Aplicada/tree/main/Semana%2011_Arquitectura%20Transformers

Herramientas y videos para entender los Transformers

-  Visualización interactiva de Transformer: bbycroft.net/llm
-  Explicación paso a paso: [YouTube - 3Blue1Brown](#)
-  DotCsv - Transformers:
- [Parte 1](#) 
- [Parte 2](#) 
-  Canal "AI by Hand": [Tom Yeh](#) 

Material complementario

Recursos técnicos adicionales

- Attention is All You Need: paperswithcode.com/paper/attention-is-all-you-need
- Deep Learning applied to NLP: arxiv.org/abs/1703.03091